Distributed Multi-Model Analytics for E-commerce Data

Technical Report - AUCA Big Data Analytics Final Project (MongoDB, HBase, Apache Spark)

Student: HITAYEZU James **Student ID:** 101381 **Date:** November 2026 **Code:** github.com/hitayezujames/ecommerce-analytics

Summary. A synthetic 90-day e-commerce platform (10,000 users, 5,000 products, 200,000 sessions with 2.0 M page views, 50,000 orders worth $53.2 M) is stored in two NoSQL systems chosen per access pattern - **MongoDB** for the catalogue, customer profiles and orders (rich, nested documents) and **HBase** for the clickstream and daily product counters (append-only, time-ordered, sparse) - and analysed with **Apache Spark**, which cleans the raw JSON into a Parquet star schema, runs batch analytics and joins the two stores. Key findings: (1) **38% of sessions flagged 'converted' have no order** in the order store; (2) **stock-outs, not popularity, explain most of the views-to-sales correlation** (r = 0.70 overall, 0.14 for in-stock products), costing ~18,600 product views; (3) abandoned carts hold **3.7x the revenue** of completed web orders; (4) Champions + Loyal customers are 52% of buyers but 68% of revenue; (5) device, referrer and product co-purchase effects are statistically indistinguishable from chance - a property of the generator that the analysis detects rather than over-interprets.

1. System Architecture

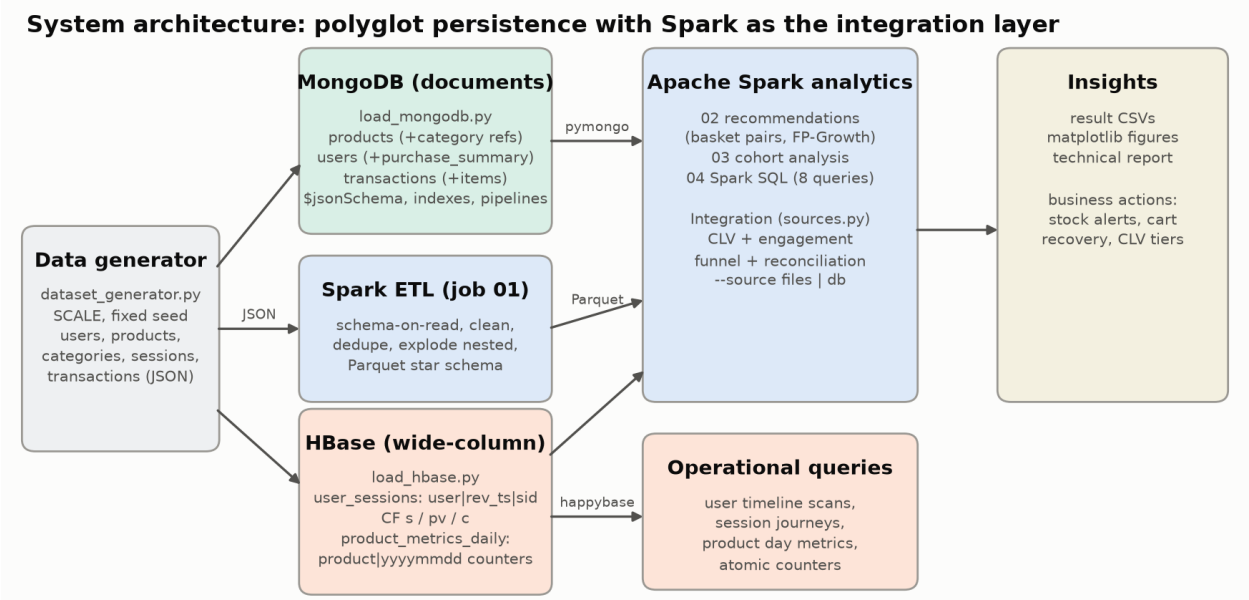


Figure 0 - Data flow. Each store holds the data whose access pattern it serves best; Spark reads either the Parquet copy or the live stores through one interface (integration/sources.py).

The generator writes JSON files that feed three paths. **MongoDB** receives categories, products, users and transactions and serves document-shaped questions (a customer's profile, an order with its lines, revenue by category). **HBase** receives every session keyed by user and reverse time plus a daily metric row per product, serving range scans over activity streams and atomic counter updates. **Spark** (PySpark 3.5) converts the raw JSON once into typed Parquet (job 01) and runs recommendations, cohorts and Spark SQL on it; the two integrated queries can read either Parquet (*--source files*) or MongoDB + HBase directly (*--source db*) with identical column contracts, so the same analytics code serves both. MongoDB and HBase (standalone, with its Thrift server) are installed by *setup/setup_ubuntu.sh*; the whole pipeline is one command: *python run_all.py*.

Dataset preparation. The supplied *dataset_generator.py* is a Colab notebook saved under a .py name; its code was extracted unchanged (kept for reference) and an improved version keeps the same page-flow model, probabilities and output format while fixing six issues: products now receive the *subcategory_id* the brief's schema requires; a fixed END_DATE and seeded IDs replace *datetime.now()/uuid4()* so runs are reproducible; a SCALE factor sizes the data; sessions are streamed to disk in 100k chunks (the original keeps 2 M sessions in RAM, > 8 GB); inactive products can no longer enter carts; and a category dict replaces a per-page linear scan. Results below use SCALE = 0.1 (200k sessions, 50k orders, 400 MB of JSON); all code runs unchanged at full scale.

2. Data Modeling and Storage (Part 1)

2.1 MongoDB document model

Schema design started from the queries: catalogue browsing by category, a customer profile page, order history, and revenue/profit aggregation. The rule applied was *embed what is read together and is bounded; reference what is shared or unbounded*.

Collection	Document shape	Pattern and justification
categories (25)	{_id, name, subcategories:[{id, name, profit_margin}]}	Hierarchy is tiny (3-5 subcategories) and read as a whole - embedded array.
products (5,000)	{_id, name, base_price, current_stock, is_active, category:{id,name}, subcategory:{id,name,profit_margin}, price_history:[{price,date}]}	**Extended reference**: category/subcategory names and margin copied in so listing and filtering need no $lookup. price_history is bounded (1-3) so it is embedded.
transactions (50,000)	{_id, user_id, session_id, timestamp:Date, status, payment_method, subtotal, discount, total, items:[{product_id, product_name, category_id, subcategory_id, profit_margin, quantity, unit_price, subtotal}]}	Line items **embedded** (always read with the order, written once). Product fields are a **snapshot at time of sale**, which is also historically correct when prices or margins change later.
users (10,000)	{_id, geo_data, registration_date, last_active, purchase_summary:{total_orders, total_spent, avg_order_value, first/last_purchase, favourite_category, recent_transactions[5]}}	**Computed + subset pattern**: a profile needs the summary and the last 5 orders, not all orders. Built server-side with $group + **$merge** after loading, so it can be refreshed incrementally.

Data quality and performance. ISO strings are stored as BSON Dates (enables $dateDiff, $dateToString and range indexes); $jsonSchema validators reject orders without items or with negative totals; inserts are unordered 5,000-document batches so one bad record does not abort a batch. Indexes follow the query patterns: {user_id, timestamp:-1} for order history, {timestamp} for period revenue, multikey {items.product_id} for 'orders containing X', a **partial** index on session_id (only web orders have one) for the funnel join, and {category.category_id, base_price} for catalogue filters. *aggregations.py --explain* prints the winning plan so index use can be verified (the 30-day revenue query should show an IXSCAN on timestamp).

Aggregation pipelines (mongodb/aggregations.py, results in data/output/mongo_*.csv):

- **Revenue analytics** - $match (status, optional date range) -> $project net factor = total/subtotal so the order-level discount is allocated pro-rata to lines -> $unwind items -> $group by category and month (net revenue, gross profit = line x margin, units, distinct orders) -> $lookup category names (25 docs) -> margin %.
- **Product popularity** - one $unwind/$group pass, then **$facet** returns top-10 by units, by revenue and by distinct buyers plus catalogue coverage in a single round-trip.
- **Customer segmentation (RFM-lite)** - per-user frequency, monetary and recency ($dateDiff to END_DATE), $switch into 7 segments, $group, and **$setWindowFields** for each segment's share of revenue (Table 1). A $bucket pipeline gives the order-frequency distribution. (Users carry no age field, so frequency/value segmentation was used instead of age groups.)

Segment (rule)	Customers	Avg orders	Avg spend	Revenue share	Avg recency (d)
Champions (>=8 orders, <=14 d)	1,050	8.9	$9,546	18.8%	5
Loyal (>=5, <=30 d)	4,138	6.0	$6,373	49.6%	11
At risk (>=5, >30 d)	427	5.9	$6,261	5.0%	39
Developing (2-4, <=30 d)	2,937	3.3	$3,549	19.6%	13
Lapsing (2-4, >30 d)	1,070	3.0	$3,121	6.3%	46
One-time (recent / inactive)	100 / 227	1.0	$1,067 / $1,173	0.2% / 0.5%	13 / 61

Table 1 - MongoDB RFM segmentation output (9,949 buyers; 51 users never ordered).

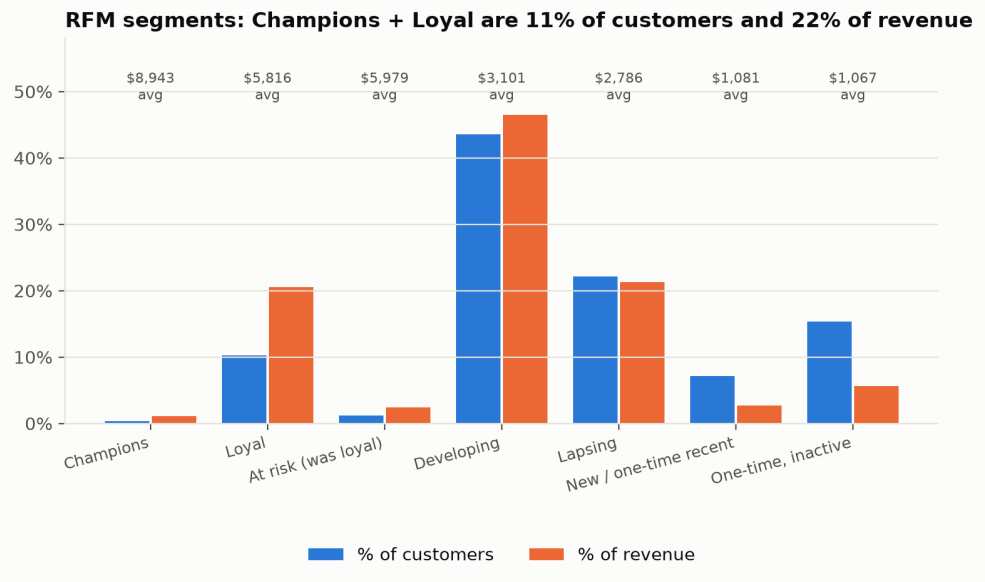


Figure 4 - RFM segments: share of customers vs share of revenue (MongoDB pipeline 3).

2.2 HBase wide-column model

HBase stores the data that is **append-only, time-ordered, high-volume and sparse**. In HBase the row key is the only index, so the design is the key design:

Table / row key	Column families	Why
user_sessions user_id | reverse_ts | session_id reverse_ts = 9999999999 - epoch(start)	**s**: 14 metadata cells (times, duration, device, referrer, status, geo, counts) - IN_MEMORY **pv**: one column per page view, qualifier 000..015, compact JSON value **c**: one column per cart line, qualifier = product_id, value qty|price	User prefix keeps a user's sessions contiguous -> one prefix scan; reverse time puts the newest first so 'last N sessions' is a scan with LIMIT N; a time window is a bounded [start,stop) range scan; session_id suffix guarantees uniqueness. Separate families mean listing sessions never reads the bulky pv files. Sessions have 3-15 views and most have no cart - wide, sparse rows cost nothing for absent cells.
product_metrics_daily product_id | yyyymmdd	**m**: views, cart_adds, purchases, units_sold, revenue_cents as 8-byte longs; TTL 400 days	A product's history is one contiguous range -> date-range scans; counters allow atomic INCREMENT from a live stream; money stored as integer cents (no float drift); TTL ages out old rows.

Both tables are **pre-split** into 5 regions on the key range (user_002000 ... / prod_01000 ...) so the bulk load and reads are spread across region servers instead of hot-spotting one region; ROW bloom filters speed point GETs; GZ compression shrinks repetitive JSON; VERSIONS = 1 because events are immutable. The loader writes through client-side batches of 2,000 puts and skips malformed rows. Implemented queries (hbase/query_hbase.py and the shell script sample_queries.hbase): Q1 latest 5 sessions of a user (prefix scan, family s, LIMIT 5); Q2 sessions in a date window (computed start/stop rows); Q3 full journey of one session (single-row GET on pv and c); Q4 a product's daily metrics for a period (range scan); Q5 weekly view-to-purchase conversion rolled up from Q4; Q6 atomic counter increment; plus a server-side SingleColumnValueFilter for converted sessions.

Hot-spot discussion. Prefixing by user spreads writes because the active user at any moment is random; a timestamp-first key would send all current writes to the last region. If a few 'whale' users dominated traffic, a 1-byte hash salt (hash(user) mod N) would be prefixed and reads fanned out over N ranges.

2.3 Why this data in MongoDB versus HBase

Criterion	MongoDB (catalogue, users, orders)	HBase (sessions, product-day metrics)
Access pattern	Lookup by id, filters on many fields, ad-hoc aggregations	Range scans by key prefix/time; point GETs
Shape	Nested, heterogeneous documents read as a unit	Wide, sparse rows; variable number of events
Write profile	Moderate, updates (stock, summaries)	Very high, append-only, counters
Secondary access	Rich secondary/multikey/partial indexes	Only the row key - designed per query
Volume (full scale)	~0.5 M orders, 15 k reference docs	2 M sessions, ~20 M page-view cells, growing daily
Consistency need	Multi-field documents, validation	Row-level atomicity is enough

3. Data Processing with Apache Spark (Part 2)

3.1 Cleaning and normalisation (01_clean_normalize.py)

Raw files are JSON *arrays*, so they are read with multiLine = true and an **explicit StructType schema** (no inference pass over ~4 GB at full scale; stable types - e.g. discount is sometimes the integer 0; missing fields such as subcategory_id in the original generator become NULL instead of failing). Steps: trim strings and turn empty strings into NULL; normalise case of enumerations (status, payment, page type, device; upper-case country codes); parse ISO strings to timestamps and derive date/month columns; drop rows without keys and **de-duplicate on business keys**; recompute session duration from end-start and order subtotal from the lines (flagging mismatches); clamp negative stock; fall back to the category's average margin when a subcategory is unknown; **explode** nested data into fact tables (posexplode page views keeps their order; the cart map becomes rows); enrich order lines with category and margin via **broadcast joins**; and allocate order discounts pro-rata to compute net revenue and gross profit per line. Output is a Parquet star schema - 4 dimensions (users, categories, subcategories, products), price history, and 5 facts (sessions with funnel flags, page views **partitioned by event_date**, cart items, transactions, transaction items) - plus a data-quality report.

Data-quality findings: no duplicates, orphans or total mismatches were found, but referential checks revealed that **15,552 of 40,749 'converted' sessions (38%) have no transaction**. Tracing the generator explains it: once the transaction target is reached, converted sessions stop producing orders (and an order is silently dropped when one cart line is out of stock). In a real system this is exactly the reconciliation gap - a tracking flag disagreeing with the order system - that the integrated funnel (Section 4.2) measures instead of trusting the flag.

Optimisation choices: explicit schemas; convert once to columnar Parquet + snappy (later jobs read only the columns they need from compressed columnar files instead of re-parsing JSON); date partitioning for pruning; broadcast joins for dimensions under 50 MB; Adaptive Query Execution (coalesces shuffle partitions, splits skewed joins); shuffle partitions sized to the machine (64 locally); caching only DataFrames reused several times.

3.2 Product recommendation indicators (02_recommendations.py)

'Bought together' pairs come from a self-join of user-level baskets on basket id with a.product < b.product (halves the pair space), keeping pairs seen by >= 2 users and computing support, confidence in both directions and lift; the per-product top-5 list ranks by confidence with a window function. 'Viewed together' does the same on product-detail views within a session, and MLlib **FP-Growth** on order baskets cross-checks the result. **Critical result:** 19,986 product pairs reach two co-buyers with an apparent mean lift of 35 - which looks like strong affinity but is not. With 5,000 products there are 12.5 M possible pairs and the expected co-occurrence per pair is only 0.055; a Poisson model predicts ~17,700 pairs with >= 2 by pure chance, close to what was observed, and no pair exceeds 4. At category level, where every pair has hundreds of observations, lift is 1.03-1.13, i.e. independence. The generator samples products uniformly, so there is no affinity to find; the pipeline (with support thresholds raised) is ready for real baskets, and reporting 'top recommendations' here would be over-fitting noise.

3.3 Cohort analysis (03_cohort_analysis.py)

Two cohort definitions are computed. **Registration cohorts** (the brief's example) group users by sign-up month and track active buyers, % of cohort buying and cumulative revenue per member by calendar month; because every registration is 90-270 days before the end date and every order is in the last 90 days, months-since-registration start at 3, so the more actionable view is the **acquisition cohort** (month of first purchase) retention matrix (Figure 5): 81-82% of each cohort buys again in each following month. A guard removes the partial trailing month (the data ends at 00:49 on 1 October), which would otherwise show a fake collapse to 0% retention.

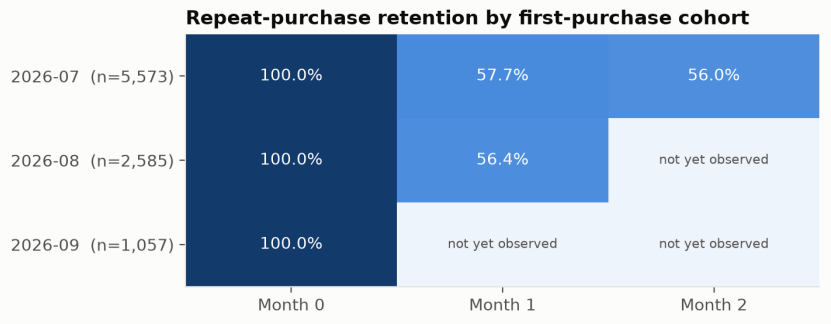


Figure 5 - Acquisition-cohort retention matrix (Spark job 03).

3.4 Spark SQL analytics (04_spark_sql_analytics.py)

Cleaned tables are registered as views and eight SQL queries use CTEs, window functions (RANK, LAG, moving averages, ROW_NUMBER), conditional aggregation and multi-way joins: (1) category revenue/profit with share and ranks; (2) daily revenue with 7-day moving average and week-over-week change; (3) conversion and revenue per session by device x referrer; (4) views vs cart adds vs units sold per product; (5) abandoned-cart value by category; (6) sales after price cuts vs rises; (7) a **cross-store** query joining *mongo_users*, *mongo_transactions* and *hbase_sessions* views to compare engagement and value by country; (8) a reconciliation of HBase 'converted' sessions against MongoDB orders. Queries 7-8 run on views that *notionally* live in the operational stores; in the integration layer the same column contracts are filled from MongoDB and HBase, so the SQL logic is unchanged.

```
WITH engagement AS (SELECT user_id, COUNT(*) sessions, AVG(duration_seconds)/60 avg_min FROM
hbase_sessions GROUP BY user_id),
     value AS (SELECT user_id, COUNT(*) orders, SUM(total) spent FROM mongo_transactions GROUP BY user_id)
SELECT u.country, COUNT(*) users, AVG(e.sessions), AVG(COALESCE(v.spent,0)), SUM(v.spent)/SUM(e.sessions)
rev_per_session
FROM mongo_users u LEFT JOIN engagement e USING (user_id) LEFT JOIN value v USING (user_id) GROUP BY
u.country
```

4. Integrated Analytics (Part 3)

4.1 Customer Lifetime Value with engagement (integration/clv_analysis.py)

Business question: which customers will be worth most over the next 12 months, how concentrated is that value, and which engaged users are under-monetised? **Sources:** MongoDB users (profile, country) and transactions with embedded lines and margins (value); HBase user_sessions, family *s* only (engagement: sessions, minutes, pages, cart sessions); Spark for joins, retention estimation and scoring. **Steps:** (1) pymongo reads with projections push field selection into MongoDB and HBase scans request one column family - only needed bytes cross the network; (2) per-user value and engagement aggregates; (3) monthly retention *r* = P(buy in month m+1 | bought in m) estimated from the data (r = 0.815); (4) $\mathbf{CLV_{12m}}$ **= monthly gross profit x** $\mathbf{SUM_{t=0..11}\ (r/(1+d))^t}$ with d = 1%/month (retention-based CLV, Gupta & Lehmann 2003) using margin rather than revenue; (5) CLV quintiles and an engagement x value quadrant around the medians.

Results: total 12-month CLV is $21.8 M (multiplier 4.78 on monthly profit). Value is only moderately concentrated - the top 20% hold 36.5% (Figure 7), far below the 60-80% typical of real retailers - and engagement barely predicts spend (corr(sessions, revenue) = 0.14). The quadrants make this actionable: 2,407 **window shoppers** visit as often as the best customers (23 sessions, 15 cart sessions) but are worth $1,275 vs $3,161, so their problem is conversion, not traffic; 2,138 **efficient buyers** are high-value with low engagement and should not be spammed. **Performance:** per-user aggregation happens before the joins (10k rows each), so the join is small; at millions of users the MongoDB Spark Connector and an HBase-Spark connector would parallelise reads across executors (supported via USE_MONGO_SPARK_CONNECTOR), and the global ntile window would be replaced with percent_rank on approximate quantiles.

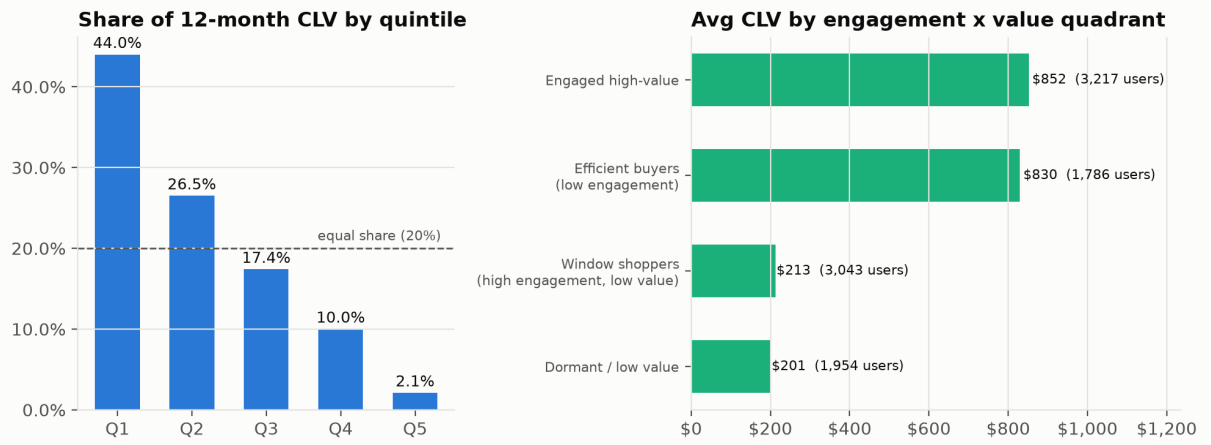


Figure 7 - CLV concentration by quintile and average CLV by engagement x value quadrant.

4.2 Conversion funnel and reconciliation (integration/funnel_analysis.py)

Business question: where do shoppers drop out, does it vary by device or channel, and do the tracking and order systems agree? Stages 1-4 (session, product view, add to cart, checkout) come from HBase - in db mode the *pv* family is scanned to derive page-type flags - and stage 5 (paid order) comes from MongoDB orders joined on session_id (partial index). Of 200,000 sessions, 98.8% view a product, 65.1% add to cart, 29.1% reach checkout with a cart and **12.6% produce an order** (Figure 3). Checkout-to-order (43%) is the weakest step but is inflated by the reconciliation gap: 70% of checkouts are flagged converted, yet 15,552 of those have no order. Conversion by device and referrer ranges 12.3-12.8% with 95% confidence intervals of +/-0.3 pp that all overlap (Figure 8) - no channel is genuinely better.

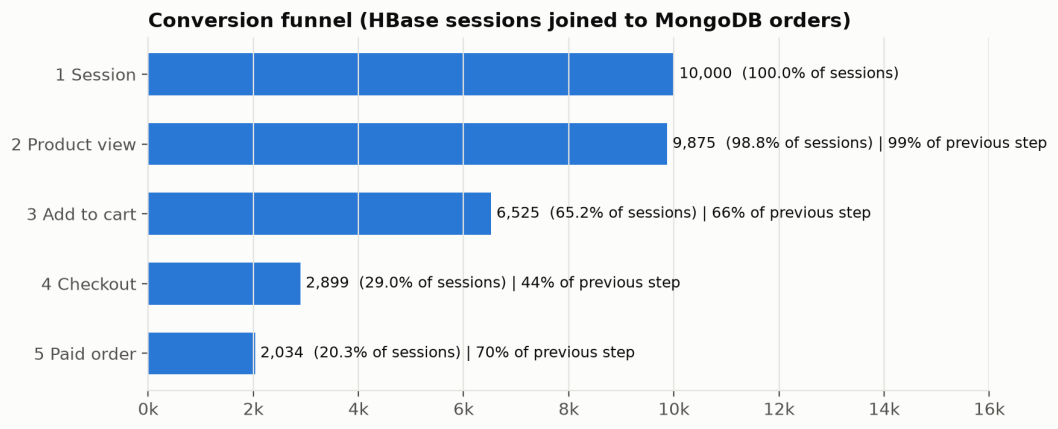


Figure 3 - Funnel across HBase (stages 1-4) and MongoDB orders (stage 5).

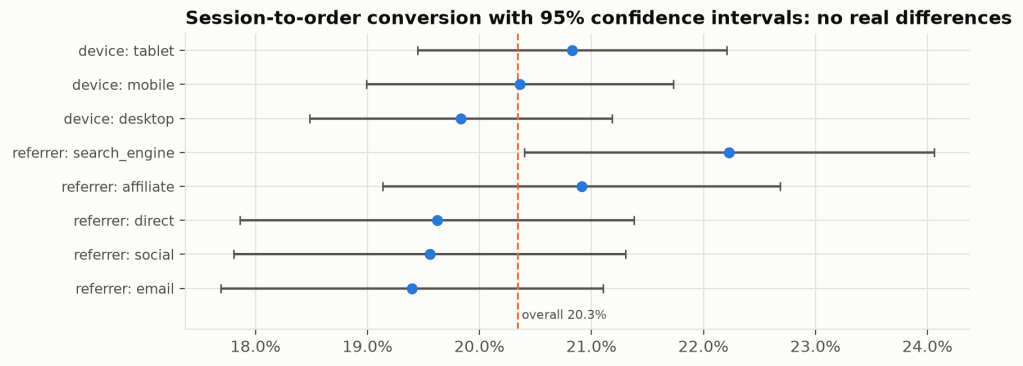


Figure 8 - Session-to-order conversion by device and referrer with 95% confidence intervals.

5. Visualizations and Business Insights (Part 4)

All static figures are produced by *visualization/make_charts.py* (matplotlib) from the result files. In addition, *visualization/build_dashboard.py* builds an optional interactive dashboard (*report/dashboard.html*, one self-contained file, no server needed) with KPI tiles, hover tooltips, switchable funnel and category views, a views-vs-sales scatter and a searchable product table.

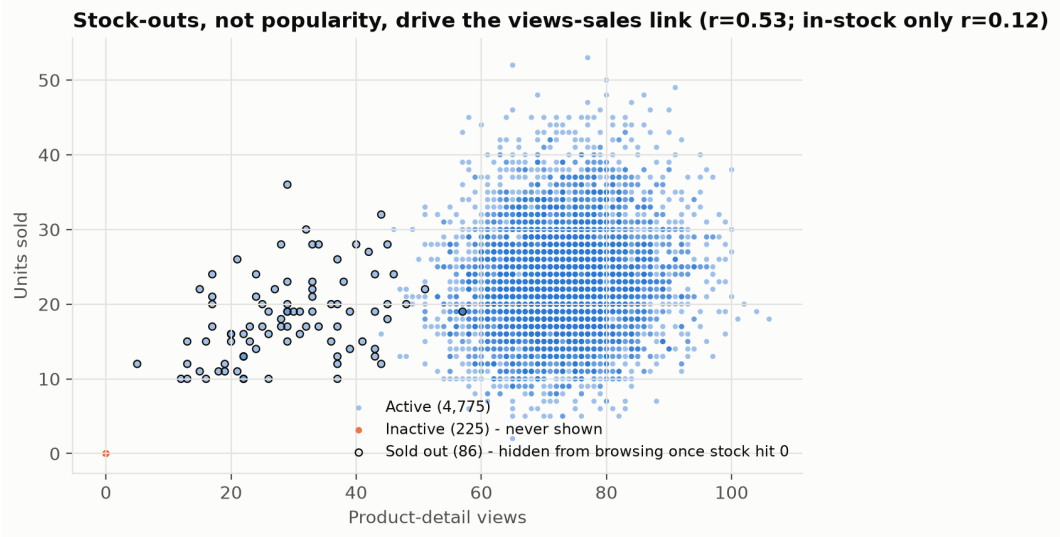


Figure 6 - Views vs units sold per product (Spark SQL query 4); sold-out products circled.

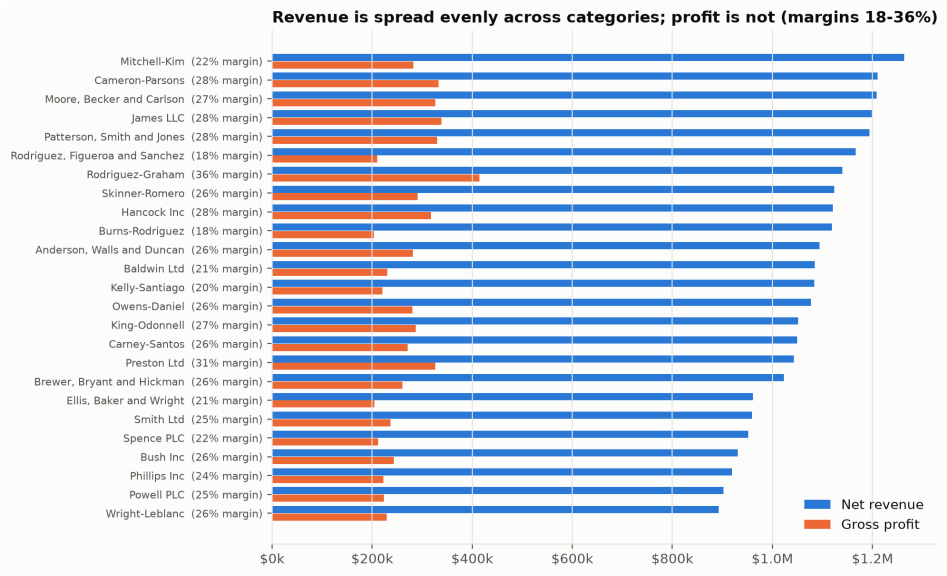


Figure 2 - Net revenue and gross profit by category (Spark SQL query 1).

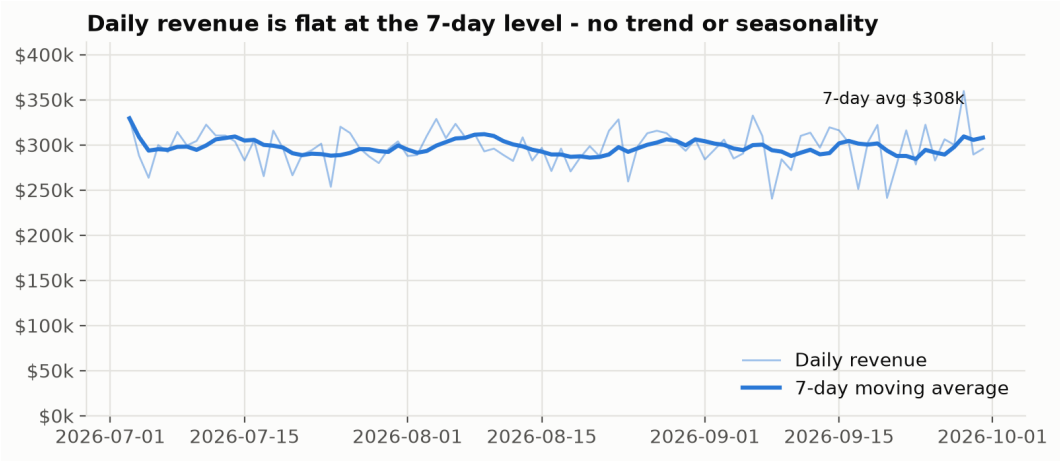


Figure 1 - Daily revenue and 7-day moving average (mean $591k/day, sd $31k; Spark SQL query 2).

Insight (evidence)	Recommended action
Stock-outs suppress demand. 190 active products sold out; they averaged 51 views vs 149 for in-stock items (~18,600 lost views). Within in-stock products views barely relate to sales (r = 0.14) (Fig. 6).	Low-stock alerts from the HBase daily counters (units_sold trend vs current_stock); reorder fast sellers; show 'back soon' pages rather than hiding products.
Abandoned carts are the largest pool of value: $71.5 M in abandoned carts vs $19.4 M of web orders; 65% of sessions build a cart, 12.6% buy (Fig. 3).	Cart-recovery e-mails within 24 h prioritised by cart value; test checkout friction (guest checkout, payment options).

Insight (evidence)	Recommended action
Tracking and order data disagree for 38% of 'converted' sessions.	Make the order store the source of truth for KPIs; add a daily reconciliation job (SQL query 8) with alerting.
Profit is less even than revenue. Categories each hold 3.3-4.6% of revenue, but margins run 18-36% (Fig. 2): Smith-Lewis Inc earns 2x the profit of Taylor-Johnson Inc on similar revenue.	Steer promotions and placement towards high-margin subcategories; review pricing of 18% categories.
Loyal + Champions = 52% of buyers, 68% of revenue; 427 'at-risk' ex-loyal customers average $6,261 (Fig. 4).	Win-back offers to the at-risk segment; VIP benefits for Champions; nurture Developing to Loyal.
Non-session orders (app/phone) have AOV $1,362 vs $771 for web orders (2.8 vs 1.6 items).	Investigate what drives larger baskets there; add bundles / free-shipping thresholds online.
No channel, device, weekday or co-purchase effect beyond chance (Fig. 1, 8; Section 3.2).	Do not reallocate budget on these differences; run controlled A/B tests before acting.

Table 2 - Key findings and actions (SCALE = 0.1 run).

6. Scalability Considerations

- **Ingestion:** the generator streams 100k-session chunks; at production scale JSON-lines (splittable) or Kafka would replace multi-line arrays, which Spark cannot split (one task per file).
- **MongoDB:** shard *transactions* on {user_id: hashed} (even writes, user-local history queries) and keep catalogue collections unsharded; the computed purchase_summary avoids scanning orders for profile reads; $merge refreshes can run incrementally on recent orders only.
- **HBase:** scales horizontally by region splits; pre-splitting and user-prefixed keys avoid hot regions; salting is the next step for skewed users; TTL bounds the metrics table; bulk loads at scale would use HFile generation from Spark instead of Thrift puts.
- **Spark:** all jobs are DataFrame/SQL (Catalyst-optimised) and run unchanged on YARN/Kubernetes by setting SPARK_MASTER; Parquet + partition pruning + broadcast joins + AQE keep shuffles small; the product-pair self-join is the only super-linear step and is capped by MAX_BASKET and min-support filters (FP-Growth is the scalable alternative).
- **Integration:** pymongo/happybase reads funnel data through the driver - fine for the subset loaded here; at scale use the MongoDB Spark Connector and hbase-spark connector so each executor reads its own shard/region.

7. Technology Selection Justification

Analytical task	Technology	Reason
Customer profile, order history, catalogue filters	MongoDB	Nested documents read as a unit; secondary indexes
Revenue / segmentation reports on orders	MongoDB aggregation	Embedded lines + snapshots: no joins needed
User activity timeline, session replay	HBase	Row-key range scans, newest first, sparse wide rows
Real-time product counters	HBase	Atomic INCREMENT, time-bucketed rows, TTL
Cleaning millions of JSON records	Spark	Parallel schema-on-read, explode, dedupe, Parquet
Recommendations, cohorts, CLV, funnel	Spark (DataFrame/SQL/MLlib)	Large joins and windows across sources

8. Limitations and Future Work

- **Synthetic data:** uniform sampling produces flat channels, no seasonality and no product affinity; the analyses are built to detect this rather than invent patterns, but business conclusions need real data.
- **HBase subset:** HBase holds a representative subset of 10,000 of the 100,000 generated sessions. The development laptop had 3.6 GB RAM (about 2.3 GB for WSL) shared by MongoDB, HBase and Spark; with a 512-700 MB HBase heap, sustained writes through the Thrift server caused broken connections, timeouts and ZooKeeper session timeouts after 20,000-30,000 sessions. The loader therefore retries failed batches, can resume (--skip) and caps the load (HBASE_MAX_SESSIONS). Sessions are generated in random order across users and the 90-day window, so the subset keeps device, referrer and page-flow distributions; it does, however, contain none of the generator's late-run 'converted without order' sessions. All Spark analyses use the full 100,000 sessions. On adequate hardware the full set would be bulk-loaded as HFiles from Spark instead of row-by-row Thrift puts.
- **Environment:** single-node HBase/MongoDB (Docker) on a 4 GB laptop do not demonstrate cluster behaviour; results use a 5% scale dataset (SCALE=0.05), while all code runs unchanged at full scale.

- **CLV model:** a single average retention rate and a 3-month window; future work: BG/NBD + Gamma-Gamma per-customer models and validation on a hold-out period.
- **Money as doubles** in MongoDB for aggregation simplicity (HBase uses integer cents); production should use Decimal128.
- **Future:** Spark Structured Streaming from Kafka into HBase counters (real-time funnel), ALS collaborative filtering once real interactions exist, and an orchestrated (Airflow) daily pipeline with data-quality gates.

Reproducibility: *bash setup/setup_ubuntu.sh; bash setup/start_services.sh; python run_all.py* - see SETUP_GUIDE.md and README.md for step-by-step commands. **Reference:** Gupta, S. & Lehmann, D. R. (2003). Customers as assets. *Journal of Interactive Marketing*, 17(1), 9-24.